01-Classical Classifier Baseline with Extra data

In this notebook, we will do a simple baseline classification using classicial method (Tfidf and logistic regression) We also add extra data from Pitt dataset

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import precision_recall_fscore_support, f1_score, classificati
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
```

In [2]:
```
# train_fname = "../data/splits/train.csv"
# dev_fname = "../data/splits/dev.csv"

# new - load 50/50 splits
train_fname = "../../data/splits/train_50pc.csv"
dev_fname = "../../data/splits/dev_50pc.csv"

train_df = pd.read_csv(train_fname)
dev_df = pd.read_csv(dev_fname)

print("Train len =", len(train_df))
print("Dev df =", len(dev_df))
```

```
Train len = 83
Dev df = 83
```

In [3]:
```
extra_train_df = pd.read_csv("../../data/splits/train_pitt.csv")
extra_train_df.head()
```

Out[3]:

	text	target
0	little boy getting in the cookie jar .\nand th...	CN
1	well ǂ the sink's overflowing .\nand &-uh the ...	CN
2	this picture ?\nwell, there's a kid on a stool...	AD
3	everything I see in action or everything I +.....	CN
4	there's a little boy or> there's a little gir...	AD

In [4]:
```
train_df.head()
```

Out[4]:

	text	target
0	I think you see going on in that picture. We'l...	AD
1	I'm gonna give you a picture that has a lot of...	AD
2	Ah the sinks running over waters come on all f...	CN
3	Well, my boys trying to get some cookies and t...	AD
4	I'd like you to do is just look at the picture...	CN

In [5]:
```
dev_df.head()
```

Out[5]:

	text	target
0	Everything you see going on in this picture, g...	CN
1	Now you just look at the picture. Okay, this i...	CN
2	Tell me everything that you see going on in th...	AD
3	Girls and boys getting her one that stores sto...	AD
4	All right, we have a few more tasks to go thro...	CN

Pre-processing

We will lowercase the text for better efficiency

In [6]:
```
extra_train_df["text_lower"] = extra_train_df["text"].apply(lambda text: text.lower
```

In [7]:
```
train_df["text_lower"] = train_df["text"].apply(lambda text: text.lower().strip())
dev_df["text_lower"] = dev_df["text"].apply(lambda text: text.lower().strip())
```

In [8]:
```
extra_train_df.head()
```

Out[8]:

	text	target	text_lower
0	little boy getting in the cookie jar .\nand th...	CN	little boy getting in the cookie jar .\nand th...
1	well ‡ the sink's overflowing .\nand &-uh the ...	CN	well ‡ the sink's overflowing .\nand &-uh the ...
2	this picture ?\nwell, there's a kid on a stool...	AD	this picture ?\nwell, there's a kid on a stool...
3	everything I see in action or everything I +.....	CN	everything i see in action or everything i +.....
4	there's a little boy or> there's a little gir...	AD	there's a little boy or> there's a little gir...

In [9]:
```
train_df.head()
```

Out[9]:

	text	target	text_lower
0	I think you see going on in that picture. We'l...	AD	i think you see going on in that picture. we'l...
1	I'm gonna give you a picture that has a lot of...	AD	i'm gonna give you a picture that has a lot of...
2	Ah the sinks running over waters come on all f...	CN	ah the sinks running over waters come on all f...
3	Well, my boys trying to get some cookies and t...	AD	well, my boys trying to get some cookies and t...
4	I'd like you to do is just look at the picture...	CN	i'd like you to do is just look at the picture...

In [10]:
```
dev_df.head()
```

Out[10]:

	text	target	text_lower
0	Everything you see going on in this picture, g...	CN	everything you see going on in this picture, g...
1	Now you just look at the picture. Okay, this i...	CN	now you just look at the picture. okay, this i...
2	Tell me everything that you see going on in th...	AD	tell me everything that you see going on in th...
3	Girls and boys getting her one that stores sto...	AD	girls and boys getting her one that stores sto...
4	All right, we have a few more tasks to go thro...	CN	all right, we have a few more tasks to go thro...

In [11]:
```
# stack up extra-train and train df

# stacked_train_df = pd.concat([train_df, extra_train_df], ignore_index=True, axis=

# test with pre-train with extra data only
stack = False  # set to True to combine train_df with extra_train_df
replace = True # set to True to replace train df with extra_train_df (Pitt dataset)

# the combination stack=False and replace=True works best, accuracy=85%, recall=97%
if replace:
    stacked_train_df = extra_train_df
elif stack:
    stacked_train_df = pd.concat([train_df, extra_train_df], ignore_index=True, axi
else:
    # keep the original train_df
    stacked_train_df = train_df

print(len(stacked_train_df))
stacked_train_df.head()
```

372

Out[11]:

	text	target	text_lower
0	little boy getting in the cookie jar .\nand th...	CN	little boy getting in the cookie jar .\nand th...
1	well ‡ the sink's overflowing .\nand &-uh the ...	CN	well ‡ the sink's overflowing .\nand &-uh the ...
2	this picture ?\nwell, there's a kid on a stool...	AD	this picture ?\nwell, there's a kid on a stool...
3	everything I see in action or everything I +.....	CN	everything i see in action or everything i +.....
4	there's a little boy or> there's a little gir...	AD	there's a little boy or> there's a little gir...

Build Tfidf Vectorizer

We can use Tfidf Vectorizer to convert text to a fixed-length vector

In [12]:
```
vectorizer = TfidfVectorizer(max_features=10000)

# corpus = train_df["text_lower"]
corpus = stacked_train_df["text_lower"]
print(len(corpus))
print("Building Tfidf Vectorizer...")
X_train = vectorizer.fit_transform(corpus)
print("Completed")
```

```
372
Building Tfidf Vectorizer...
Completed
```

In [13]:
```
print(X_train.shape)
```

```
(372, 1490)
```

Train Logistic regression using Tfidf vectorizer

We can now train a simple logistic regression classifier using processed X_train

In [14]:
```
X_train = vectorizer.fit_transform(stacked_train_df["text_lower"])
y_train = stacked_train_df["target"]

print("Training logistic classifier...")
# clf = LogisticRegression(random_state=42, max_iter=200).fit(X_train, y_train)
clf = LogisticRegression(random_state=42,
                         max_iter=500,
                        ).fit(X_train, y_train)
print("Completed")
```

```
Training logistic classifier...
Completed
```

In [15]:
```
# getting score on train set
train_score = clf.score(X_train, y_train)
print(f"Train accuracy = {train_score*100:.2f} %")
```

```
Train accuracy = 90.86 %
```

Get accuracy using cross validation

In [16]:
```
print("Performing cross-validation...")
scores = cross_val_score(clf, X_train, y_train, cv=10)
print("Completed")
print(scores)
print("Average =", np.average(scores))
```

```
Performing cross-validation...
Completed
[0.89473684 0.84210526 0.7027027  0.78378378 0.86486486 0.78378378
 0.89189189 0.81081081 0.89189189 0.89189189]
Average = 0.8358463726884781
```

In [17]:
```
# get precision / recall / f1
y_pred = clf.predict(X_train)
print(classification_report(y_train, y_pred, target_names=["AD", "CN"]))
```

```
              precision    recall  f1-score   support

          AD       0.90      0.94      0.92       205
          CN       0.92      0.87      0.90       167

    accuracy                           0.91       372
   macro avg       0.91      0.90      0.91       372
weighted avg       0.91      0.91      0.91       372
```

Now we can evaluate on dev set

In [18]:
```
X_dev = vectorizer.transform(dev_df["text_lower"])
y_dev = dev_df["target"]
print("X_dev shape =", X_dev.shape)
print("y_dev len =", len(y_dev))
```

```
X_dev shape = (83, 1490)
y_dev len = 83
```

In [19]:
```
dev_score = clf.score(X_dev, y_dev)
print(f"Dev accuracy = {dev_score*100:.2f} %")
```

```
Dev accuracy = 77.11 %
```

In [20]:
```
y_pred = clf.predict(X_dev)
print(classification_report(y_dev, y_pred, target_names=["AD", "CN"]))
```

```
                precision    recall  f1-score   support

          AD       0.73      0.94      0.82        47
          CN       0.87      0.56      0.68        36

    accuracy                           0.77        83
   macro avg       0.80      0.75      0.75        83
weighted avg       0.79      0.77      0.76        83
```

The baseline result has improved with additional data from Pitt:

- Accuracy improved to 78% compared with 74% in previous baseline
- Recall improved to 89%, compared with 76% in previous baseline
- Average 10-fold accuracy improved to 80% from 76%

If we pre-train with Pitt dataset only, without using our train set at all:

- the accuracy dropped slightly from 79% to 77%
- but recall improved from 90% to 94%

In [21]:
```
### Process prediction for test set
```

In [22]:
```
test_fname = "../../data/splits/test_text.csv"
test_df = pd.read_csv(test_fname)
print("Test df =", len(test_df))
test_df["text_lower"] = test_df["text"].apply(lambda text: text.lower().strip())
test_df.head()
```

```
Test df = 71
```

Out[22]:

	ID	Prediction	text	text_lower
0	adrsdt15	NaN	I see oh boy, almost falling on a stool trying...	i see oh boy, almost falling on a stool trying...
1	adrsdt40	NaN	Well the kids are trying to get the cookies an...	well the kids are trying to get the cookies an...
2	adrsdt26	NaN	Okay, this is a family setting in a kitchen an...	okay, this is a family setting in a kitchen an...
3	adrsdt67	NaN	Oh, what do I see in this picture? This remind...	oh, what do i see in this picture? this remind...
4	adrsdt58	NaN	While the young boy is reaching for a cookie j...	while the young boy is reaching for a cookie j...

In [23]:
```
X_test = vectorizer.transform(test_df["text_lower"])
test_df["Prediction"] = clf.predict(X_test)
test_df.head()
```

Out[23]:

	ID	Prediction	text	text_lower
0	adrsdt15	AD	I see oh boy, almost falling on a stool trying...	i see oh boy, almost falling on a stool trying...
1	adrsdt40	CN	Well the kids are trying to get the cookies an...	well the kids are trying to get the cookies an...
2	adrsdt26	CN	Okay, this is a family setting in a kitchen an...	okay, this is a family setting in a kitchen an...
3	adrsdt67	AD	Oh, what do I see in this picture? This remind...	oh, what do i see in this picture? this remind...
4	adrsdt58	AD	While the young boy is reaching for a cookie j...	while the young boy is reaching for a cookie j...

In [24]:
```
# convert to 0/1 to be consistent to ground truth
label2binary = lambda label: "0" if label=="0" or label.lower().strip() == "cn" els
assert label2binary("CN") == "0"
assert label2binary("AD") == "1"
assert label2binary("0") == "0"
assert label2binary("1") == "1"

print("Passed")
```

```
Passed
```

In [25]:
```
# apply to prediction column
test_df["Prediction"] = test_df["Prediction"].apply(label2binary)
# write to prediction output file
output_path = "../../evaluation/"
fname = "test_results_task1_classical_text_Pitt_replace.csv"
output_df = test_df.drop(columns=["text", "text_lower"])
output_df.head()
```

Out[25]:

	ID	Prediction
0	adrsdt15	1
1	adrsdt40	0
2	adrsdt26	0
3	adrsdt67	1
4	adrsdt58	1

In [26]:
```
output_df.to_csv(output_path + fname,
                 index=False
                )
print("Completed")
```

```
Completed
```

Output from evaluate script (stack=False, replace=True)

```
$ python evaluate.py task1_gold.csv test_results_task1_classical_text_Pitt_replace.csv
Loading data from task1_gold.csv...done
Loading data from test_results_task1_classical_text_Pitt_replace.csv...done
Gold len = 71
Prediction len = 71
Classification report:
              precision    recall  f1-score   support

           0       0.96      0.72      0.83        36
           1       0.77      0.97      0.86        35

    accuracy                           0.85        71
   macro avg       0.87      0.85      0.84        71
weighted avg       0.87      0.85      0.84        71

Accuracy = 0.8450704225352113
Recall = 0.9714285714285714
```

Evaluation output for replace=False, stack=True

```
$ python evaluate.py task1_gold.csv test_results_task1_classical_text_Pitt_comb.csv
Loading data from task1_gold.csv...done
Loading data from test_results_task1_classical_text_Pitt_comb.csv...done
Gold len = 71
Prediction len = 71
Classification report:
              precision    recall  f1-score   support

           0       0.86      0.69      0.77        36
           1       0.74      0.89      0.81        35

    accuracy                           0.79        71
   macro avg       0.80      0.79      0.79        71
weighted avg       0.80      0.79      0.79        71

Accuracy = 0.7887323943661971
Recall = 0.8857142857142857
```